

Intro to Computer Vision With Deep Learning

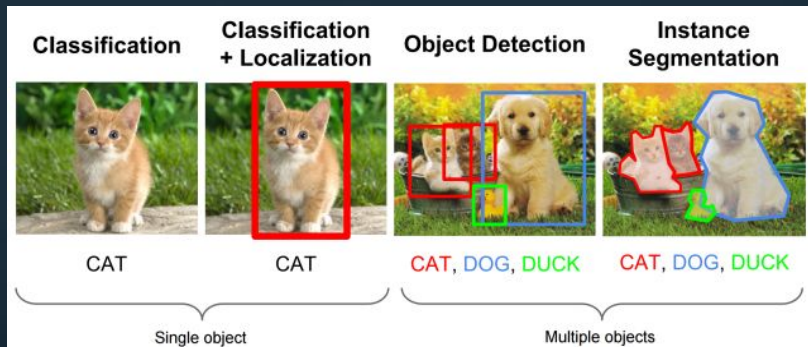
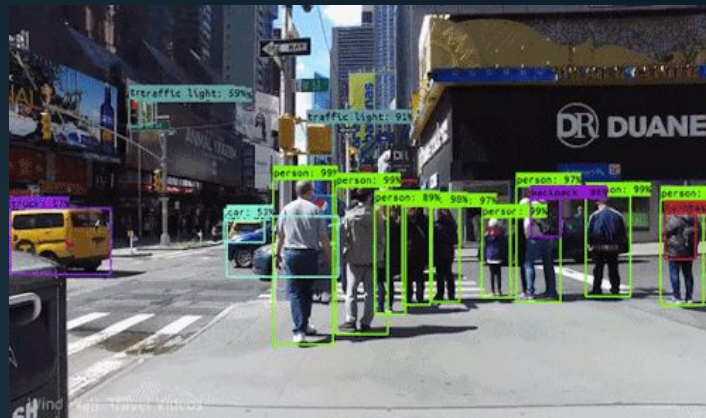
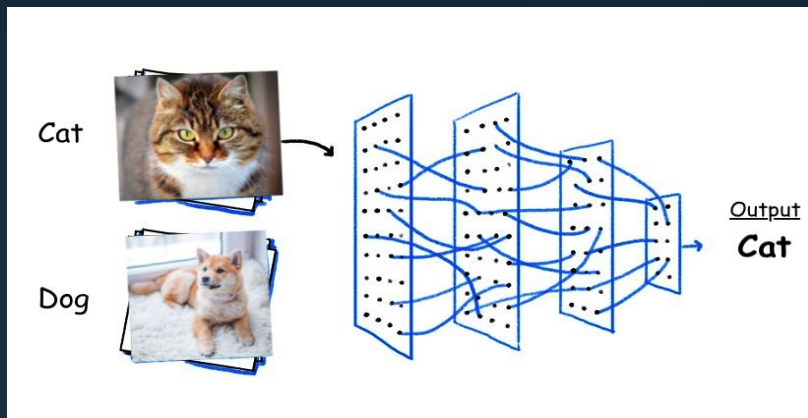


Agenda

- Intro to Computer Vision
- Computer Vision vs. Human Vision
- Computer Vision Challenges
- Computer Vision with AI
- Convolutional Neural Network
- Transfer Learning

Intro

Computer Vision



Computer Vision vs. Image Processing

Image Processing

Transforming the content in some way
(such as brightness , color or cropping).

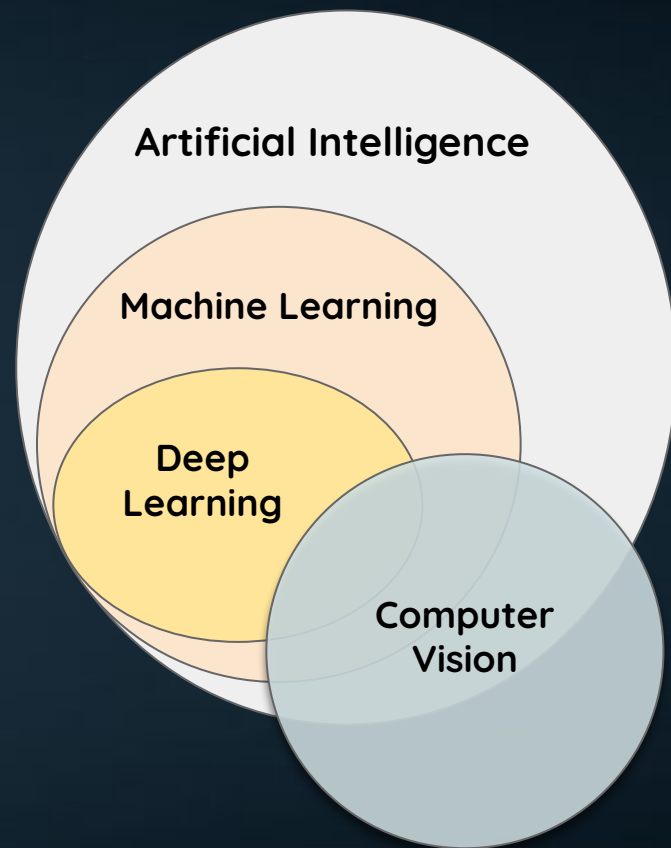
- Not concerned with understanding the content of an image.
- A computer vision system may require image processing to be applied to raw input, so we may say image processing is a subset of computer vision.

Computer Vision

Help computers to “**see**” and
“**understand**” the content of images.

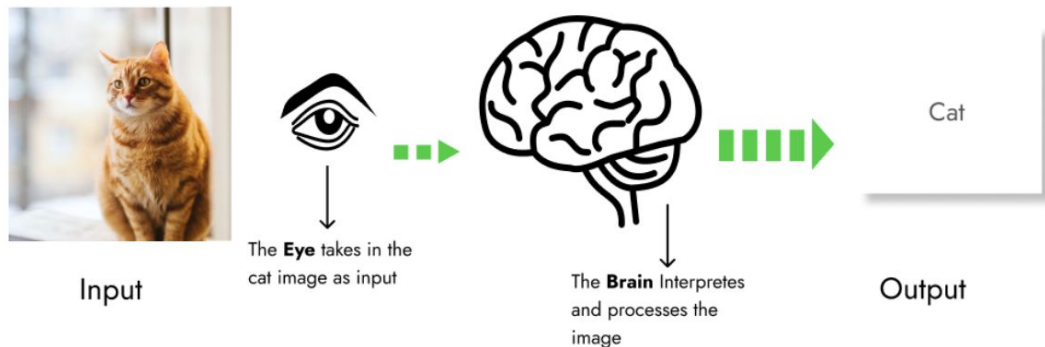
Computer Vision vs AI vs ML

- Computer vision is closely linked with machine learning.
- Machine learning is used to train computer vision models to recognize and interpret visual data, while computer vision is used to extract meaningful information from images and videos that can be used to improve machine learning models.

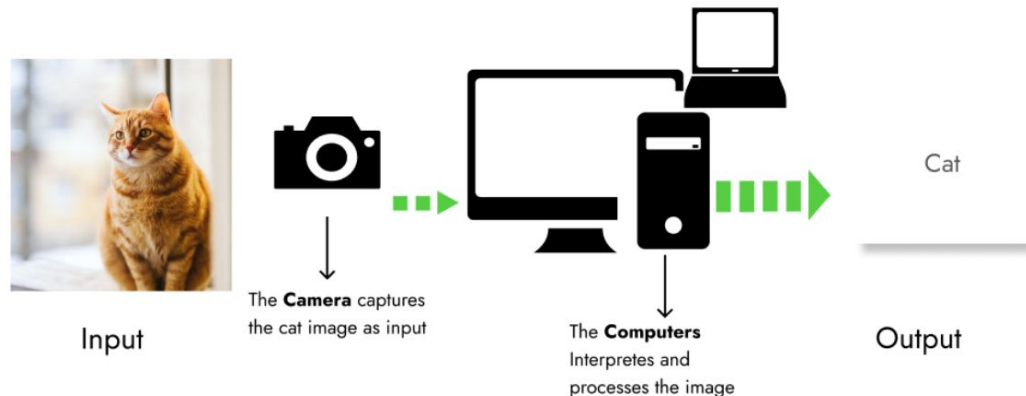


Computer Vision vs. Human Vision

Human Vision



Computer Vision



Some Computer Vision Applications

- **X-ray Analysis**

It may be used successfully in the context of medical X-ray imaging for treatment and research, MRI reconstruction, and surgical planning.

- **Agriculture**

It enables continuous real-time monitoring of plant development and identifying agricultural alterations caused by malnutrition or disease.

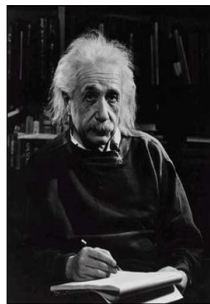
- **Self-checkout**

Autonomous check-out is now feasible owing to computer vision-based systems analyzing consumer interactions and tracking goods movement.

<https://www.xenonstack.com/blog/computer-vision-applications>



Computer Vision Challenges



What we see

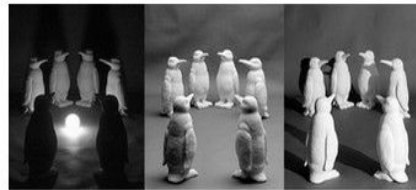
0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

What a computer sees

Viewpoint variation



Illumination conditions



Scale variation



Deformation



Occlusion



Background clutter



Intra-class variation



Image Classification

Let's Try !

Image Classification with Classic ML and MLP

Convolutional Neural Network

Convolution

- Reduce the image into a form which is easier to process, without losing features
- The idea here is that some convolutions will change the image in such a way that certain features in the image get emphasized.

Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

Image

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

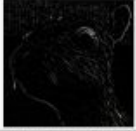
Convolved
Feature

4		

Convolution

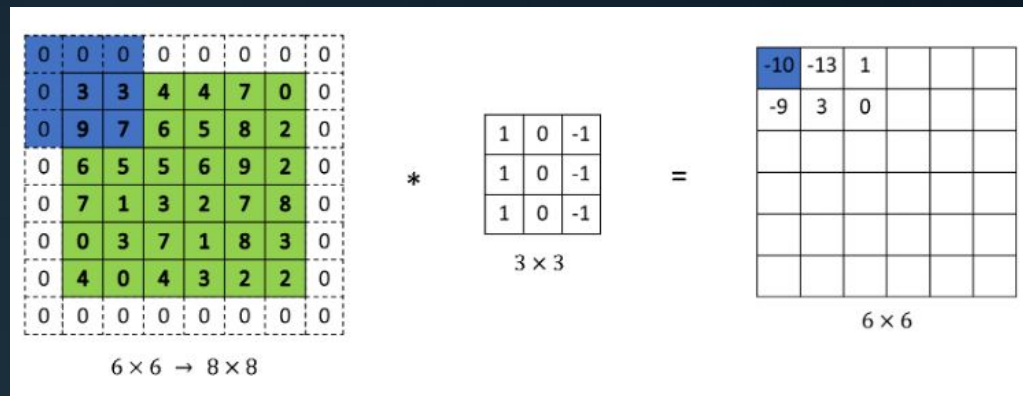
- We will have a set of filters in each CONV layer, and each of them will produce a separate activation map. We will stack these activation maps and produce the output (Output Depth = No. of filters).
- The reduction in the size of the input to the feature map is referred to as *border effects*.
- This is often not a problem for large images and small filters but can be *a problem with small images*, specially when convolutional layers are stacked.

[Try Filters](#) (Very Nice!)

Operation	Filter	Convolved Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Convolution: Padding

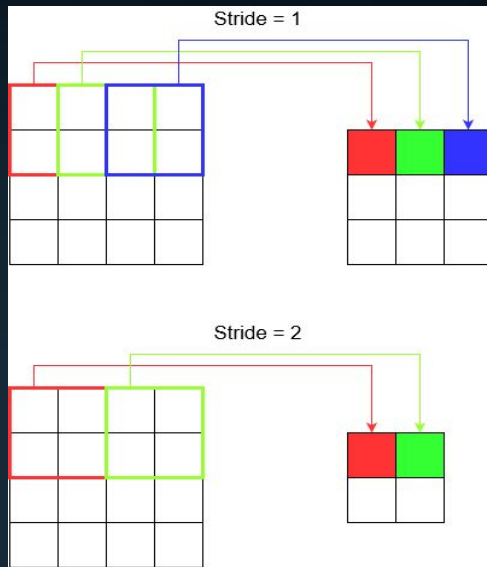
- The addition of pixels to the edge of the image is called ***padding***.



- This ensures that each pixel has an opportunity to be at the center of the filter, better for interacting, ***better for features to be detected by the filter.***
- Also, this ensures that ***the output has the same shape as the input.***

Convolution: Stride

- The filter is moved across the image left to right, top to bottom with a step. This is referred to the ***stride***.
- The stride can be changed, which has an effect on how the filter is applied to the image and, in turn, the ***size of the resulting feature map***.

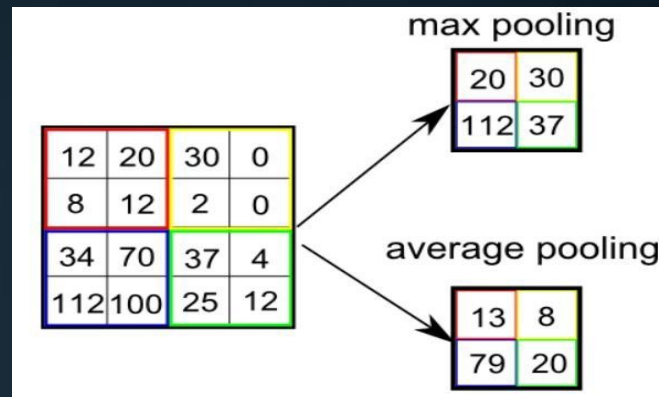


Final Notes on Conv. Layer

- Kernel (filter) values are ***random initialized weights*** for the convolution layer and will be changing during training.
- Relu will be applied to the convolution layer, ***remove non-positive values and add non-linearity***.

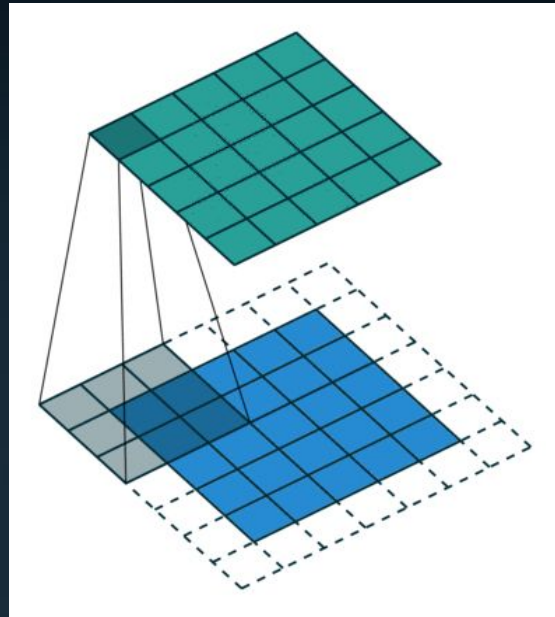
Pooling (Downsampling)

- Pooling is a way of *compressing an image*.
- This allows the network to train much faster, *focusing on the most important information* in each feature, while *decreasing the computational power* required to process the data.
- This also reduces the amount of parameters and hence *control overfitting*.
- *Max Pooling* also performs as a *Noise Suppressant*
- Max Pooling performs a lot better than Average Pooling.



Convolution and Pooling Again

- ConvNets need not be limited to only one Convolutional Layer.
- Conventionally, the first Conv Layer is responsible for capturing the Low-Level features such as edges, color, etc.
- With added layers, the architecture *adapts to the High-Level features* as well, giving us the wholesome understanding of images.

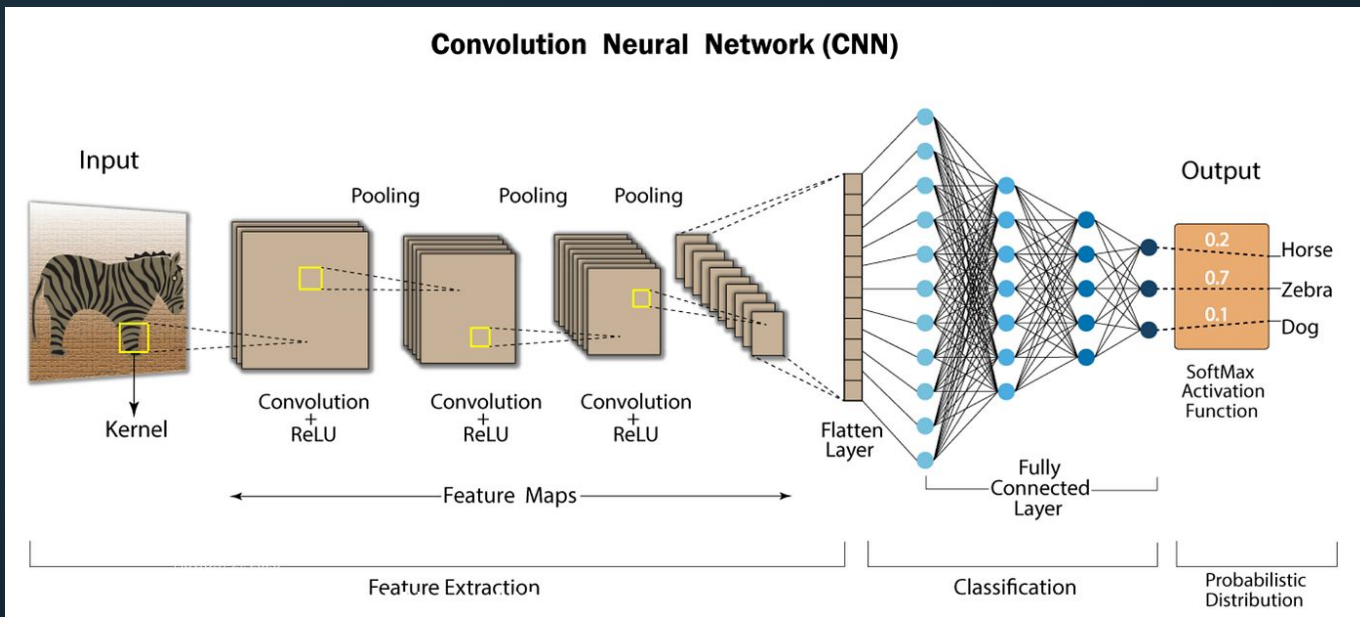


3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Flattening

- After converting our input image into a suitable form for our regular neural network, we shall flatten the image into a column vector to be fed to the regular neural network to classify the image.



Convolutional Neural Networks

- Convolutional Neural Networks (CNNs) are a type of deep learning neural network architecture that is particularly well suited to image classification and object recognition tasks.
- A CNN works by transforming an input image into a feature map, which is then processed through multiple convolutional and pooling layers to be fed to the regular neural network to produce a predicted output.
- Although CNN is essentially famous with Computer Vision Tasks, It may be used in Natural Language Processing Tasks.

Nice Article: [What is Convolutional Neural Network — CNN](#)



First Convolution NN

Tensorflow Tutorial

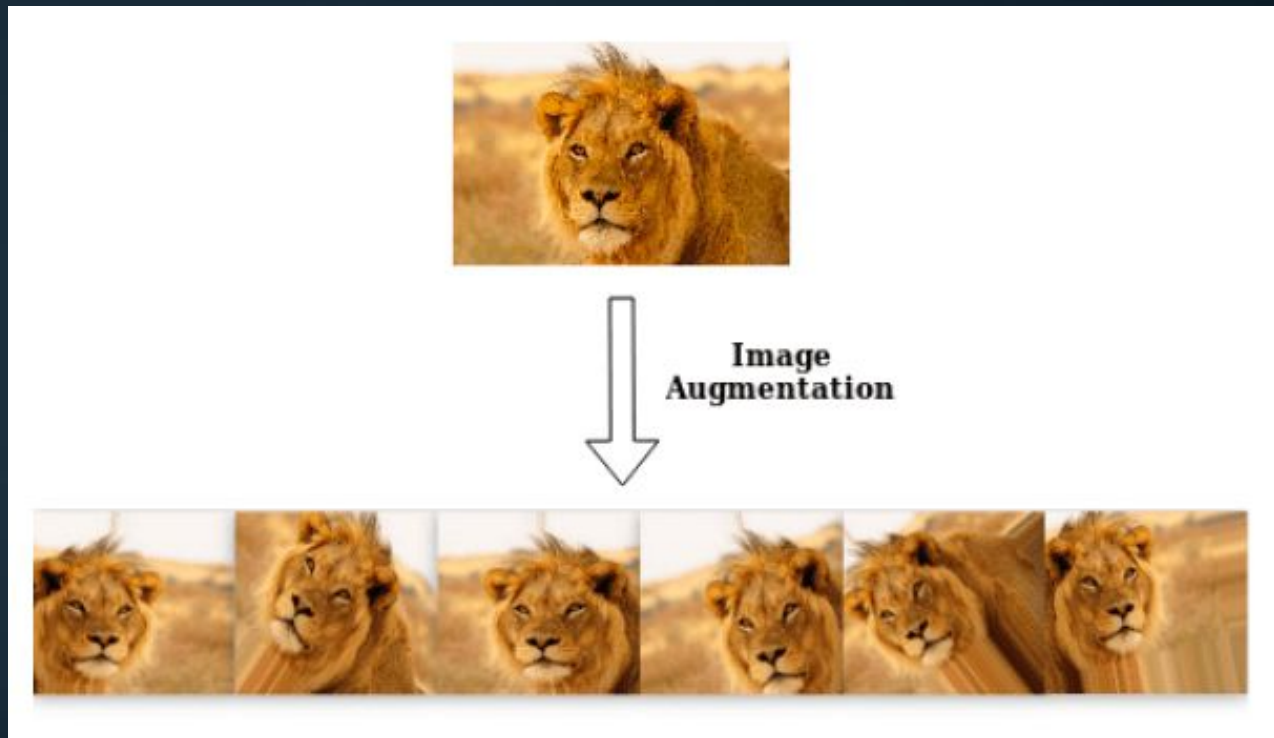
Image Classification With CNN



Practical Tips

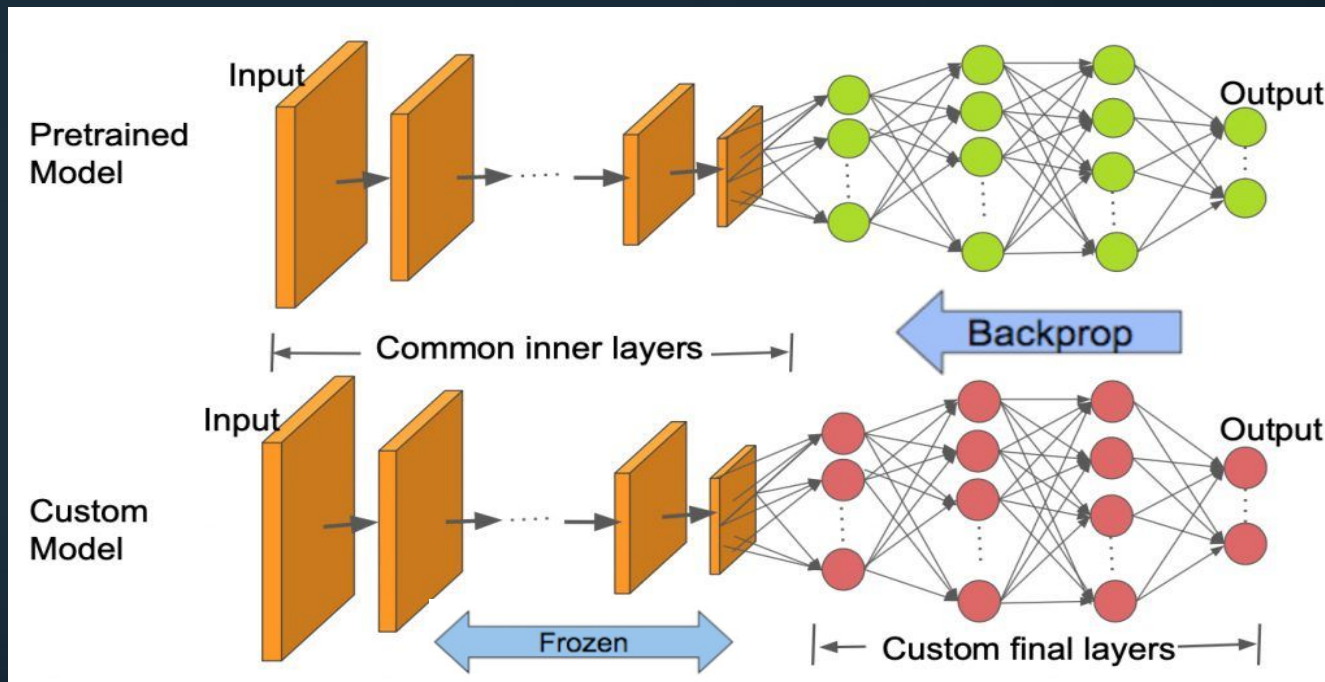
Image Augmentation

- Enlarge dataset
- Avoid Overfitting



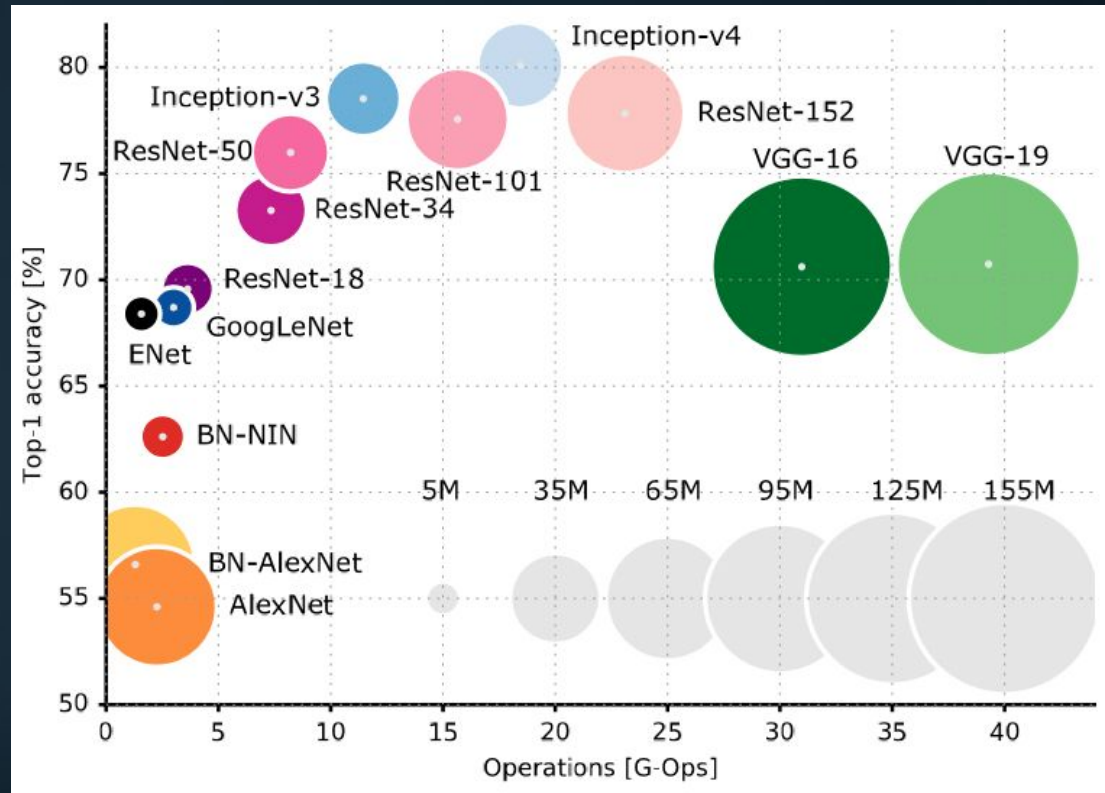
Transfer Learning

Features are more generic in early layers and more original-dataset-specific in later layers.



Popular CNN Architectures

- The vertical axis shows top 1 accuracy on ImageNet classification.
- The horizontal axis shows the number of operations needed to classify an image.
- Circle size is proportional to the number of parameters in the network.





DNN Training Optimization

- We have covered:
 - Weight Initialization.
 - Activation Functions.
 - Batch Normalization.
 - Different Optimizers.
 - Different Loss Functions.
 - Different Metrics.
 - Regularization Techniques (L1, L2, Dropout, Early Stopping)
 - Callbacks (Model Checkpoint, Early Stopping)

<https://playground.tensorflow.org/>



